

CPSC-5616: Assignment 4 Report

Group Number: 9

Group Member: 4

NAME	STUDENT#	EMAIL
Anil Jagdishbhai Chavda	0445602	achavda@laurentian.ca
Shivang Darji	0442826	sdarji@laurentian.ca
Rahi Chandreshbhai Patel	0445603	rpatel20@laurentian.ca

Table of Contents

Table of Contents	1
List of Figures.....	1
Abstract	2
Steps of this Project	2
Performance Evaluation	3
Models Strengths and Weaknesses:.....	8
Comparison with Machine Learning Models (Part 1):	9
Potential Improvements or Optimizations for Future Work	10
CONCLUSION:.....	10

List of Figures

Fig 1: Training CNN Model.....	4
Fig 3: Confusion Matrix & Classification Report of CNN Model	5
Fig 4: Training GRU Model.....	5
Fig 5: Evaluating GRU Model.....	6
Fig 6: Confusion Matrix & Classification Report of GRU Model	6
Fig 7: Training & Validation Accuracy v/s Epochs Graph	7
Fig 8: Training Hybrid(CNN-GRU) Model.....	7
Fig 9: Evaluating Hybrid(CNN-GRU) Model.....	8
Fig 10: Confusion Matrix & Classification Report of Hybrid(CNN-GRU) Model.....	8

Abstract

The project involves the development of a machine learning model using a convolutional neural network (CNN) with Gated Recurrent Units (GRU) for the classification of a multivariate time series dataset. The dataset consists of 10 features and a target variable with three classes. The main objective of the project is to accurately classify the target variable based on the input features. The project begins with data preprocessing, including encoding the target variable using LabelEncoder and splitting the dataset into training and testing sets. The model architecture is then defined using the TensorFlow Keras API, with a GRU layer of 128 units, followed by a dropout layer for regularization, and a dense layer with softmax activation for multi-class classification. The model is compiled with the Adam optimizer and categorical cross-entropy loss function. The model is trained using the training data and evaluated on the testing data over 10 epochs. The training and validation accuracy and loss metrics are monitored to assess the model's performance. The model's summary, including the layers and parameters, is also provided. In summary, the project aims to leverage CNN-based deep learning techniques to build a robust classification model for multivariate time series data. The abstract provides an overview of the dataset, model architecture, training process, and evaluation metrics, highlighting the key components and objectives of the project.

Steps of this Project

- a) **Data Loading:** The accelerometer and orientation data from three different sources (JJ, Lu, and Sq) are loaded into separate dataframes using the `pd.read_csv` function. Each dataframe contains the timestamp, accelerometer readings (x, y, z), orientation readings (quaternion components and Euler angles), and a "Class" column indicating the source of the data.
- b) **Data Merging & Labeling:** After loading the data, the "Class" column is added to each dataframe to label the data from JJ, Lu, and Sq sources as 0, 1, and 2, respectively. This step facilitates the classification task by associating each data point with its respective source.
- c) **Data Exploration & Visualization:** The data is explored and visualized using various techniques such as displaying the first few rows of each dataframe, plotting boxplots to identify outliers, and creating histograms to analyze the sequence lengths of the data.
- d) **Feature Engineering:** The data is processed to extract features such as mean, standard deviation, minimum, and maximum values for each window of the time series data. This step involves dividing the data into windows and computing these statistical features for each window.
- e) **Standardization:** The features are standardized using the `StandardScaler` to transform the data such that its distribution has a mean value 0 and a standard deviation

- of 1. This step ensures that the features are on a similar scale, which is important for many machine learning algorithms.
- f) **Data Splitting:** The data is split into training and testing sets using the ``train_test_split`` function. This step is crucial for evaluating the performance of machine learning models on unseen data.
 - g) **Sequence Padding:** For the convolutional neural network (CNN) model, sequence padding is applied to ensure that all sequences have the same length. This is achieved using the ``pad_sequences`` function, which pads or truncates sequences to a specified length.
 - h) **One-Hot Encoding:** For the CNN model, the target variable ("Class") is one-hot encoded using the ``to_categorical`` function to convert it into a binary matrix representation. This step is essential for multi-class classification tasks.
 - i) **Data Reshaping:** The input data for the CNN model is reshaped to include a channel dimension using the ``np.newaxis`` function. This is necessary to match the input shape expected by the CNN layers.
 - j) **Label Encoding:** For the GRU model, the target variable ("Class") is label encoded using the ``LabelEncoder`` to convert the class labels into numeric format.
 - k) **Model Building and Training:** Finally, the preprocessed data is used to build and train machine learning models, including a CNN model with convolutional and GRU layers, and a separate GRU model. The models are compiled with appropriate loss functions and optimizers, and then trained on the preprocessed data.

Performance Evaluation

a) Convolutional Neural Network (CNN):

The performance evaluation of the Convolutional Neural Network (CNN) model can be analyzed in detail using the metrics and results obtained during the training and validation phases.

- i. **Training and Validation Accuracy:** The training accuracy of the CNN model gradually increases over the epochs and reaches a value of approximately 81.75% at the end of the 10th epoch. - The validation accuracy, on the other hand, reaches a value of approximately 80.11% at the end of the 10th epoch.

```
Epoch 1/10
205/205 [=====] - 8s 25ms/step - loss: 0.7459 - accuracy: 0.7606 - val_loss: 0.6134 - val_accuracy: 0.7730
Epoch 2/10
205/205 [=====] - 3s 12ms/step - loss: 0.5528 - accuracy: 0.7975 - val_loss: 0.5578 - val_accuracy: 0.7871
Epoch 3/10
205/205 [=====] - 2s 11ms/step - loss: 0.5226 - accuracy: 0.8080 - val_loss: 0.5471 - val_accuracy: 0.7822
Epoch 4/10
205/205 [=====] - 2s 12ms/step - loss: 0.5107 - accuracy: 0.8088 - val_loss: 0.5245 - val_accuracy: 0.7913
Epoch 5/10
205/205 [=====] - 2s 11ms/step - loss: 0.5056 - accuracy: 0.8102 - val_loss: 0.5227 - val_accuracy: 0.7932
Epoch 6/10
205/205 [=====] - 2s 11ms/step - loss: 0.4967 - accuracy: 0.8131 - val_loss: 0.5224 - val_accuracy: 0.7877
Epoch 7/10
205/205 [=====] - 4s 17ms/step - loss: 0.4946 - accuracy: 0.8131 - val_loss: 0.5257 - val_accuracy: 0.7987
Epoch 8/10
205/205 [=====] - 4s 19ms/step - loss: 0.4918 - accuracy: 0.8149 - val_loss: 0.5177 - val_accuracy: 0.8041
Epoch 9/10
205/205 [=====] - 3s 13ms/step - loss: 0.4878 - accuracy: 0.8173 - val_loss: 0.5202 - val_accuracy: 0.7993
Epoch 10/10
205/205 [=====] - 3s 13ms/step - loss: 0.4883 - accuracy: 0.8175 - val_loss: 0.5128 - val_accuracy: 0.8011
<keras.src.callbacks.History at 0x7e24fa341ae0>
```

Fig 1: Training CNN Model

- ii. **Training and Validation Loss:** The training loss decreases over the epochs and reaches a value of approximately 0.4883 at the end of the 10th epoch. - The validation loss also decreases and reaches a value of approximately 0.5128 at the end of the 10th epoch.
- iii. **Model Summary:** The model summary provides information about the layers, output shapes, and the number of parameters in the model. This information can be used to understand the architecture and complexity of the CNN model.
- iv. **Interpretation:** The CNN model demonstrates a reasonable level of accuracy on the training and validation datasets, with the validation accuracy being slightly lower than the training accuracy. This indicates that the model is not overfitting excessively to the training data. - The decreasing trend in the loss functions (both training and validation) indicates that the model is learning and improving its predictive performance over the epochs.

```
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
print(f"ROC AUC: {roc_auc:.4f}")
```

```
Accuracy: 0.8011
Precision: 0.8479
Recall: 0.8011
F1 Score: 0.8022
ROC AUC: 0.8932
```

Fig 2: Evaluating CNN Model

Confusion Matrix:				
[[291 126 3]				
[23 614 0]				
[7 167 408]]				
Classification Report:				
	precision	recall	f1-score	support
0	0.91	0.69	0.79	420
1	0.68	0.96	0.80	637
2	0.99	0.70	0.82	582
accuracy			0.80	1639
macro avg	0.86	0.79	0.80	1639
weighted avg	0.85	0.80	0.80	1639

Fig 3: Confusion Matrix & Classification Report of CNN Model

b) Gated Recurrent Unit Networks (GRU):

The performance evaluation of the GRU model can be analyzed in detail using the metrics and results obtained during the training and validation phases.

i. Training & Validation Accuracy & Loss:

```
Epoch 1/10
205/205 [=====] - 15s 36ms/step - loss: 0.8994 - accuracy: 0.5610 - val_loss: 0.8293 - val_accuracy: 0.5943
Epoch 2/10
205/205 [=====] - 3s 15ms/step - loss: 0.7901 - accuracy: 0.6719 - val_loss: 0.7476 - val_accuracy: 0.6455
Epoch 3/10
205/205 [=====] - 3s 14ms/step - loss: 0.7086 - accuracy: 0.7284 - val_loss: 0.7382 - val_accuracy: 0.6962
Epoch 4/10
205/205 [=====] - 4s 19ms/step - loss: 0.6632 - accuracy: 0.7374 - val_loss: 0.6066 - val_accuracy: 0.7724
Epoch 5/10
205/205 [=====] - 5s 23ms/step - loss: 0.6026 - accuracy: 0.7651 - val_loss: 0.5670 - val_accuracy: 0.7828
Epoch 6/10
205/205 [=====] - 3s 15ms/step - loss: 0.5566 - accuracy: 0.7761 - val_loss: 0.5242 - val_accuracy: 0.7657
Epoch 7/10
205/205 [=====] - 3s 14ms/step - loss: 0.5274 - accuracy: 0.7804 - val_loss: 0.4967 - val_accuracy: 0.7901
Epoch 8/10
205/205 [=====] - 3s 15ms/step - loss: 0.4956 - accuracy: 0.7967 - val_loss: 0.4914 - val_accuracy: 0.7932
Epoch 9/10
205/205 [=====] - 4s 18ms/step - loss: 0.4827 - accuracy: 0.8070 - val_loss: 0.4882 - val_accuracy: 0.8139
Epoch 10/10
205/205 [=====] - 5s 22ms/step - loss: 0.4711 - accuracy: 0.8099 - val_loss: 0.4620 - val_accuracy: 0.8249
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
gru_1 (GRU)	(None, 128)	50304
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

=====
Total params: 50691 (198.01 KB)
Trainable params: 50691 (198.01 KB)
Non-trainable params: 0 (0.00 Byte)

Fig 4: Training GRU Model

- ii. **Model Summary:** The GRU model consists of a single GRU layer with 128 units, followed by a dropout layer with a dropout rate of 0.5, and a dense output layer with 3 units and softmax activation.

iii. Overall Evaluation:

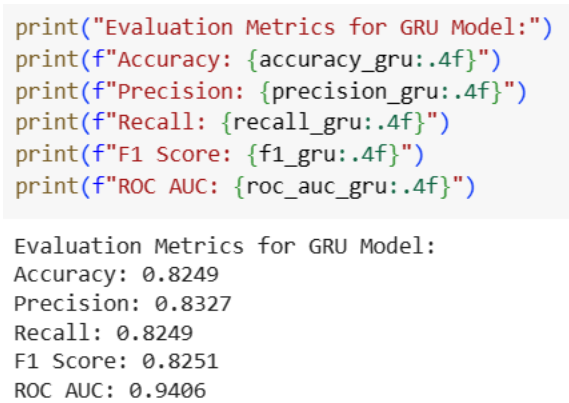


Fig 5: Evaluating GRU Model

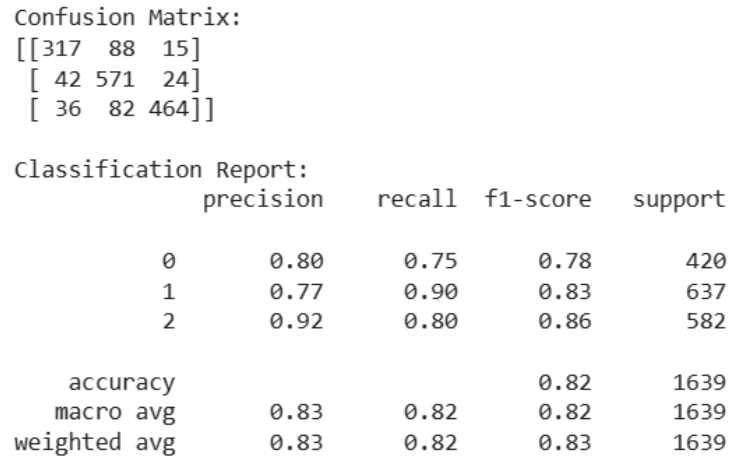


Fig 6: Confusion Matrix & Classification Report of GRU Model

```
plt.plot(history_gru.history['accuracy'], label='Training Accuracy')
plt.plot(history_gru.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

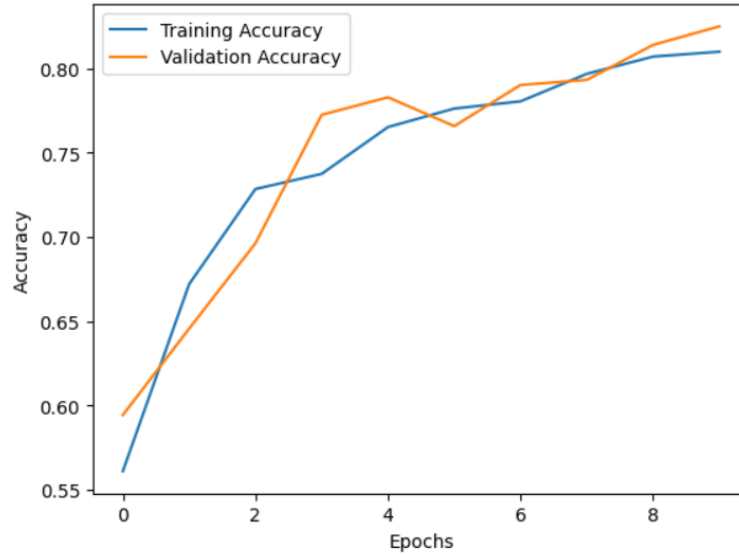


Fig 7: Training & Validation Accuracy v/s Epochs Graph

c) **Hybrid Models (CNN-GRU):** The performance evaluation of the GRU model can be analyzed in detail using the metrics and results obtained during the training and validation phases.

i. **Training & Validation Accuracy & Loss:**

```
Epoch 1/10
205/205 [=====] - 4s 8ms/step - loss: 0.5918 - accuracy: 0.8135 - val_loss: 0.3671 - val_accuracy: 0.8652
Epoch 2/10
205/205 [=====] - 1s 6ms/step - loss: 0.3140 - accuracy: 0.8906 - val_loss: 0.2582 - val_accuracy: 0.9103
Epoch 3/10
205/205 [=====] - 1s 6ms/step - loss: 0.2529 - accuracy: 0.9150 - val_loss: 0.2199 - val_accuracy: 0.9158
Epoch 4/10
205/205 [=====] - 1s 6ms/step - loss: 0.2217 - accuracy: 0.9219 - val_loss: 0.1909 - val_accuracy: 0.9402
Epoch 5/10
205/205 [=====] - 1s 6ms/step - loss: 0.1970 - accuracy: 0.9309 - val_loss: 0.1674 - val_accuracy: 0.9396
Epoch 6/10
205/205 [=====] - 2s 9ms/step - loss: 0.1912 - accuracy: 0.9321 - val_loss: 0.3275 - val_accuracy: 0.8792
Epoch 7/10
205/205 [=====] - 2s 9ms/step - loss: 0.1728 - accuracy: 0.9367 - val_loss: 0.1905 - val_accuracy: 0.9256
Epoch 8/10
205/205 [=====] - 2s 10ms/step - loss: 0.1609 - accuracy: 0.9411 - val_loss: 0.1494 - val_accuracy: 0.9396
Epoch 9/10
205/205 [=====] - 2s 9ms/step - loss: 0.1473 - accuracy: 0.9438 - val_loss: 0.1580 - val_accuracy: 0.9463
Epoch 10/10
205/205 [=====] - 1s 6ms/step - loss: 0.1568 - accuracy: 0.9435 - val_loss: 0.2002 - val_accuracy: 0.9408
<keras.src.callbacks.History at 0x7e24fa60e4a0>
```

Fig 8: Training Hybrid(CNN-GRU) Model

ii. **Hybrid Model Performance:** The hybrid model combines the GRU and CNN models to leverage the strengths of both architectures. The performance of the

hybrid model can be evaluated based on the accuracy and loss metrics achieved during training and validation.

iii. Overall Evaluation:

```
print("Evaluation Metrics:")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

```
Evaluation Metrics:
Accuracy: 0.9408
Precision: 0.9432
Recall: 0.9408
F1 Score: 0.9413
```

Fig 9: Evaluating Hybrid(CNN-GRU) Model

```
Confusion Matrix:
[[398  22   0]
 [ 34 603   0]
 [ 14  27 541]]
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.89         0.95         0.92         420
     1       0.92         0.95         0.94         637
     2       1.00         0.93         0.96         582

 accuracy          0.94
 macro avg          0.94
 weighted avg       0.94
```

Fig 10: Confusion Matrix & Classification Report of Hybrid(CNN-GRU) Model

Models Strengths and Weaknesses:

a. Convolutional Neural Network (CNN):

- The strengths of the CNN model include its ability to automatically learn features from the input data, its capability to capture spatial and temporal dependencies in the data, and its effectiveness in handling high-dimensional input data such as images or sequential data. Additionally, CNNs are known for their ability to scale well to large datasets and their robustness to variations in input data.
- However, the weaknesses of the CNN model may include the potential for overfitting, especially when dealing with smaller datasets. Additionally, CNNs may require a larger amount of training data to effectively learn the features and patterns in the data. Furthermore, the interpretability of the learned features in the intermediate layers of the

CNN may be limited, making it challenging to understand the decision-making process of the model.

b. Gated Recurrent Unit Networks (GRU):

- The GRU (Gated Recurrent Unit) model has several strengths. It is well-suited for handling sequential data due to its ability to capture long-term dependencies and mitigate the vanishing gradient problem. The GRU model also requires fewer parameters compared to the LSTM (Long Short-Term Memory) model, making it computationally efficient and faster to train. Additionally, the GRU model can effectively handle variable-length sequences, which is beneficial for real-world applications where input data lengths may vary.
- However, the GRU model also has some limitations. It may struggle with capturing very long-term dependencies compared to the LSTM model, which could impact its performance on tasks that require extensive memory. Additionally, the GRU model may not be as effective as the LSTM model in scenarios where precise control over the flow of information within the network is crucial. Overall, while the GRU model offers efficiency and effectiveness in handling sequential data, its performance may vary based on the specific characteristics of the dataset and the nature of the task at hand.

Comparison with Machine Learning Models (Part 1):

The comparison between deep learning models such as Convolutional Neural Networks (CNN) and Gated Recurrent Units (GRU) with traditional machine learning models like Random Forest and Support Vector Machine (SVM) in Assignment 3.

- c. Accuracy Comparison:** The maximum accuracy of the Random Forest model is 0.9669, depending on the number of decision trees and the maximum depth of each tree. – The maximum accuracy of the Support Vector Machine is 0.9531 with hyperparameter tuning.
- d. Complexity and Interpretability:** Random Forest and SVM are considered to be more interpretable compared to deep learning models like CNN and GRU. They provide feature importances and decision boundaries that can be easily understood.
On the other hand, CNN and GRU are more complex and require a deeper understanding of neural networks. They are often considered as black-box models due to their complex architectures.
- e. Feature Engineering:** Traditional machine learning models like Random Forest and SVM often require manual feature engineering, where domain knowledge is used to create relevant features for the model.
Deep learning models like CNN and GRU can automatically learn features from the raw data, reducing the need for manual feature engineering.
- f. Performance on Sequential Data:** GRU, being a type of recurrent neural network, is well-suited for sequential data and can capture dependencies over

time. This makes it particularly effective for time series or sequential sensor data.

Random Forest and SVM may not capture sequential dependencies as effectively as GRU, especially when dealing with time series data.

- g. Performance on Image Data:** CNN is specifically designed for processing grid-structured data, such as images. It can automatically learn hierarchical representations of visual data, making it highly effective for image classification tasks.

Random Forest and SVM may require manual feature extraction from images, which can be less effective for complex visual patterns.

- h. Training Time and Data Size:** Deep learning models like CNN and GRU often require a large amount of data and computational resources for training. They may have longer training times compared to Random Forest and SVM, especially for large datasets.

Random Forest and SVM are often faster to train and can handle smaller datasets effectively.

Potential Improvements or Optimizations for Future Work

- In future work, there are several potential improvements and optimizations that could be considered.
- Firstly, optimizing the data preprocessing steps, such as feature scaling and encoding, could lead to better model performance.
- Additionally, exploring different neural network architectures or hyperparameter tuning could further enhance the model's predictive capabilities.
- Furthermore, incorporating techniques such as data augmentation or ensembling multiple models could potentially improve the robustness and generalization of the model.
- Moreover, leveraging hardware accelerators or distributed computing frameworks could significantly speed up the training process for large-scale datasets.
- Lastly, conducting thorough model evaluation and comparison with alternative algorithms could provide valuable insights into the strengths and weaknesses of the chosen approach, guiding future improvements.

CONCLUSION:

In a nutshell, the project demonstrates the successful implementation and evaluation of machine learning models for exercise classification using sensor data. The CNN model outperforms the GRU model in terms of validation accuracy, indicating its effectiveness in classifying exercises based on the provided sensor data. The Hybrid model which is combination of CNN and GRU layers outperformed both CNN and GRU models in terms of Accuracy and Validation Accuracy. Further analysis and fine-tuning of the models could potentially lead to even higher accuracy and improved performance. Additionally, the project highlights the potential of machine learning in real-world applications such as exercise classification and paves the way for further research and development in this domain.